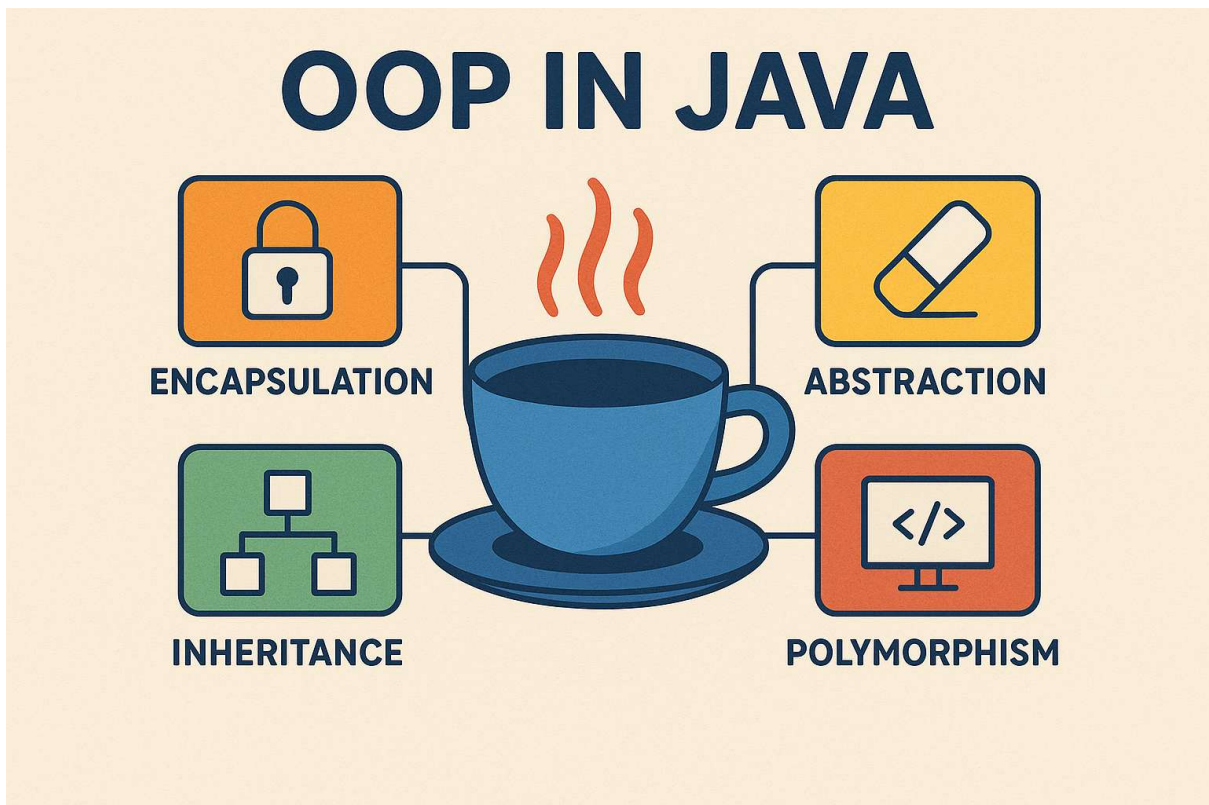


4 đặc tính OOP: đóng gói – kế thừa – đa hình – trừu tượng

Lập trình hướng đối tượng (Object-Oriented Programming - OOP) là mô hình phát triển phần mềm chủ đạo hiện nay, đặc biệt là trong ngôn ngữ Java. Sức mạnh của OOP được xây dựng dựa trên 4 nguyên lý nền tảng, hay còn gọi là "bốn trụ cột", bao gồm: Đóng gói, Kế thừa, Đa hình và Trừu tượng.

Việc hiểu sâu sắc 4 đặc tính này là yêu cầu bắt buộc đối với bất kỳ lập trình viên Java nào để xây dựng các hệ thống phần mềm an toàn, linh hoạt và dễ bảo trì.



1. Tính Đóng gói (Encapsulation)

Định nghĩa

Tính đóng gói là kỹ thuật gói gọn dữ liệu (thuộc tính) và mã nguồn xử lý dữ liệu (phương thức) vào trong một đơn vị duy nhất là Lớp (Class). Quan trọng hơn, nó là cơ chế che giấu trạng thái thực sự của đối tượng và ngăn chặn việc truy cập trực tiếp hoặc sửa đổi dữ liệu từ bên ngoài một cách tùy tiện.

Cơ chế hoạt động trong Java

Trong Java, tính đóng gói được thực hiện thông qua các bước:

1. **Khai báo thuộc tính là `private`:** Ngăn chặn các lớp khác truy cập trực tiếp vào biến.
2. **Cung cấp phương thức `public` (Getter/Setter):** Tạo ra các cổng giao tiếp công khai để đọc hoặc ghi dữ liệu. Tại đây, ta có thể đặt các logic kiểm tra (validation) để đảm bảo tính toàn vẹn của dữ liệu.

Ví dụ minh họa

```
public class SinhVien {  
    // 1. Dữ liệu được đóng gói (Private)  
    private double diemTrungBinh;  
  
    // 2. Phương thức truy cập công khai (Public)  
    public double getDiemTrungBinh() {  
        return diemTrungBinh;  
    }  
  
    public void setDiemTrungBinh(double diem) {  
        // Kiểm soát dữ liệu đầu vào  
        if (diem >= 0 && diem <= 10) {  
            this.diemTrungBinh = diem;  
        } else {  
            System.out.println("Điểm không hợp lệ!");  
        }  
    }  
}
```

2. Tính Kế thừa (Inheritance)

Định nghĩa

Tính kế thừa cho phép một lớp mới (được gọi là Lớp con/Subclass) tận dụng lại các thuộc tính và phương thức của một lớp đã tồn tại (được gọi là Lớp cha/Superclass).

Nguyên lý này giúp loại bỏ mã nguồn lặp lại (redundant code) và tạo ra cấu trúc phân cấp, thể hiện mối quan hệ "là một" (Is-A relationship). Ví dụ: Một "Sinh viên" *là một* "Con người".

Cơ chế hoạt động trong Java

- Sử dụng từ khóa `extends`.
- Java chỉ hỗ trợ đơn kế thừa (một lớp con chỉ có đúng một lớp cha trực tiếp).
- Lớp con có thể mở rộng (thêm thuộc tính/phương thức mới) hoặc định nghĩa lại hành vi (Ghi đè - Override) của lớp cha.

Ví dụ minh họa

```
// Lớp cha
class Ngươi {
    String ten;
    void an() {
        System.out.println("Đang ăn...");
    }
}

// Lớp con kế thừa từ Ngươi
class SinhVien extends Ngươi {
    String maSinhVien;
    void hoc() {
        System.out.println("Đang học bài...");
    }
}
```

3. Tính Đa hình (Polymorphism)

Định nghĩa

Đa hình (Polymorphism) xuất phát từ tiếng Hy Lạp, nghĩa là "nhiều hình thái". Trong lập trình, nó cho phép một hành động có thể được thực hiện theo nhiều cách khác nhau tùy thuộc vào đối tượng đang thực thi hành động đó.

Phân loại trong Java

Tính đa hình trong Java được chia làm hai loại chính:

A. Đa hình lúc biên dịch (Compile-time Polymorphism)

Thể hiện qua kỹ thuật **Nạp chồng phương thức (Method Overloading)**.

- Cùng tên phương thức nhưng khác nhau về số lượng hoặc kiểu dữ liệu của tham số.
- Xảy ra trong cùng một lớp.

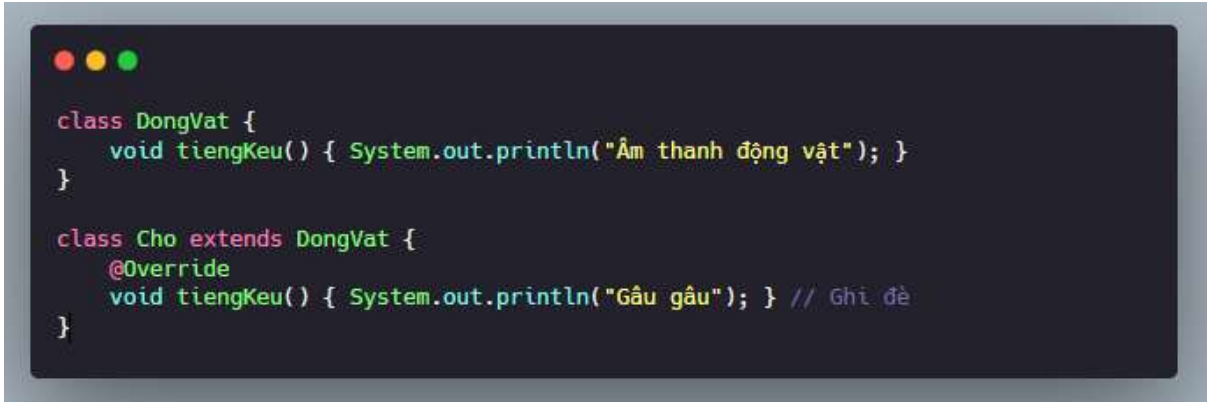
```
class TinhToan {
    // Nạp chồng phương thức tinhTong
    int tinhTong(int a, int b) { return a + b; }
    double tinhTong(double a, double b) { return a + b; }
}
```

B. Đa hình lúc chạy (Runtime Polymorphism)

Thể hiện qua kỹ thuật **Ghi đè phương thức (Method Overriding)**.

- Lớp con định nghĩa lại phương thức đã có ở lớp cha (giữ nguyên tên, kiểu trả về và tham số).

- Java Virtual Machine (JVM) sẽ quyết định gọi phương thức nào dựa trên đối tượng thực tế được tạo ra trong quá trình chạy (Runtime).



```
class DongVat {  
    void tiengKeu() { System.out.println("Âm thanh động vật"); }  
}  
  
class Cho extends DongVat {  
    @Override  
    void tiengKeu() { System.out.println("Gâu gâu"); } // Ghi đè  
}
```

4. Tính Trừu tượng (Abstraction)

Định nghĩa

Tính trừu tượng là quá trình ẩn đi các chi tiết thực thi phức tạp bên dưới và chỉ cung cấp cho người dùng những tính năng cần thiết. Nó giúp lập trình viên tập trung vào việc đối tượng "làm gì" (What) thay vì "làm như thế nào" (How).

Ví dụ: Khi lái xe ô tô, bạn chỉ cần biết đạp chân ga là xe chạy (What), không cần biết động cơ đốt trong hoạt động chi tiết ra sao (How).

Cơ chế hoạt động trong Java

Java cung cấp hai công cụ chính để thực hiện tính trừu tượng:

1. Lớp trừu tượng (Abstract Class):

- Được khai báo với từ khóa **abstract**.
- Có thể chứa phương thức trừu tượng (không có thân hàm) và phương thức thường.
- Không thể khởi tạo đối tượng trực tiếp.

2. Giao diện (Interface):

- Được khai báo với từ khóa **interface**.
- Được xem như một bản thiết kế mẫu (blueprint) thuần túy.
- Mặc định các phương thức là trừu tượng (trước Java 8). Lớp thực thi Interface phải định nghĩa lại toàn bộ các phương thức đó.

Ví dụ minh họa (Sử dụng Interface)

```
interface HìnhHoc {  
    void tinhDienTich(); // Phương thức trừu tượng: chỉ định nghĩa tên, không có  
    code xử lý  
}  
  
class HìnhTron implements HìnhHoc {  
    @Override  
    public void tinhDienTich() {  
        System.out.println("Tính diện tích theo công thức:  $PI * r * r$ ");  
    }  
}
```

Tổng kết: Bốn đặc tính trên kết hợp chặt chẽ với nhau tạo nên sức mạnh của Java, cho phép xây dựng các ứng dụng quy mô lớn, dễ dàng nâng cấp và làm việc nhóm hiệu quả.